

```
In [ ]: import os
import numpy as np
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt
```

```
In [ ]: path = os.path.abspath(os.getcwd())
file_path = os.path.join(os.path.dirname(path), "data", "rent_prediction.csv")

df = pd.read_csv(file_path)
```

Dimensions, colonnes et types

```
In [55]: numeric_cols = list(df.drop("IdentifiantMaison", axis=1).select_dtypes(include=["number"]).columns)
category_cols = list(df.drop("IdentifiantMaison", axis=1).select_dtypes(include=["object", "str"]).columns)

print(f"Colonnes dans les données : {list(df.columns)}")
print()
print(f"La forme de données du DataSet : {df.shape}")
print()
print(f"Type de données du DataSet : {df.info()}")
```

Colonnes dans les données : ['IdentifiantMaison', 'Chambres', 'Salon', 'SalleDeBainInterieure', 'Parking', 'Meuble', 'Jardin', 'Superficie_m2', 'DistanceRoute_m', 'Quartier', 'AgeMaison', 'LoyerMensuel_BIF']

La forme de données du DataSet : (510, 12)

```
<class 'pandas.DataFrame'>
RangeIndex: 510 entries, 0 to 509
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   IdentifiantMaison      510 non-null   int64
1   Chambres               485 non-null   float64
2   Salon                  485 non-null   str
3   SalleDeBainInterieure  485 non-null   str
4   Parking                485 non-null   str
5   Meuble                 485 non-null   str
6   Jardin                 485 non-null   str
7   Superficie_m2          485 non-null   float64
8   DistanceRoute_m        485 non-null   float64
9   Quartier               485 non-null   str
10  AgeMaison              485 non-null   float64
11  LoyerMensuel_BIF       510 non-null   int64
dtypes: float64(4), int64(2), str(6)
memory usage: 58.8 KB
Type de données du DataSet : None
```

Premier coup d'oeil aux données

```
In [13]: df.head()
         df.tail()
         df.sample(10)
```

Out[13]:

	house_id	rooms	living_room	indoor_bathroom	parking	Meuble	Jardin	Superficie_m2	dstance_to_road	Quartier	h
478	479	3.0	Oui	Oui	Non	Non	Oui	164.0	300.0	Kiriri	
121	122	1.0	Oui	Non	Oui	Non	Non	36.0	49.0	Nyakabiga	
247	248	5.0	Oui	Oui	Non	Non	Oui	241.0	205.0	Ngagara	
361	362	5.0	Oui	Oui	Oui	Non	Oui	NaN	33.0	Rohero	
228	229	4.0	Oui	Oui	Oui	Non	Oui	208.0	186.0	Gasekebuye	
289	290	4.0	Oui	Oui	Oui	Non	Non	187.0	298.0	Kinanira	
59	60	4.0	Oui	Oui	NaN	Non	Non	NaN	209.0	Kiriri	
287	288	NaN	Oui	Oui	Non	Oui	Non	164.0	228.0	Gasekebuye	
310	311	3.0	Oui	Non	Oui	Non	Oui	119.0	26.0	Cibitoke	
259	260	1.0	Non	Oui	Oui	NaN	Non	55.0	224.0	Bwiza	

Statistiques descriptives

In [14]: `df.describe()`
`# df.describe(include=["object"])`

Out[14]:

	house_id	rooms	Superficie_m2	house_age	monthly_rent
count	510.000000	485.000000	485.000000	4.850000e+02	5.090000e+02
mean	255.500000	3.521649	168.762887	5.380724e+03	1.241945e+06
std	147.368586	1.707436	79.610256	1.180590e+05	6.985334e+05
min	1.000000	1.000000	30.000000	0.000000e+00	1.361490e+05
25%	128.250000	2.000000	103.000000	1.000000e+01	6.991380e+05
50%	255.500000	4.000000	171.000000	2.000000e+01	1.077747e+06
75%	382.750000	5.000000	236.000000	3.000000e+01	1.678140e+06
max	510.000000	6.000000	320.000000	2.600000e+06	2.600000e+06

Identification de valeurs manquantes

```
In [15]: df.isnull()  
df.isnull().sum().sort_values(ascending=False)
```

```
Out[15]: rooms                25  
living_room                 25  
indoor_bathroom            25  
Meuble                     25  
Jardin                     25  
Superficie_m2              25  
distance_to_road           25  
Quartier                   25  
house_age                  25  
parking                    24  
monthly_rent                1  
house_id                   0  
dtype: int64
```

Lignes en double

```
In [17]: df.duplicated()  
df.duplicated().sum()
```

```
Out[17]: 0      False  
1      False  
2      False  
3      False  
4      False  
...  
505    False  
506    False  
507    False  
508    False  
509    False  
Length: 510, dtype: bool
```

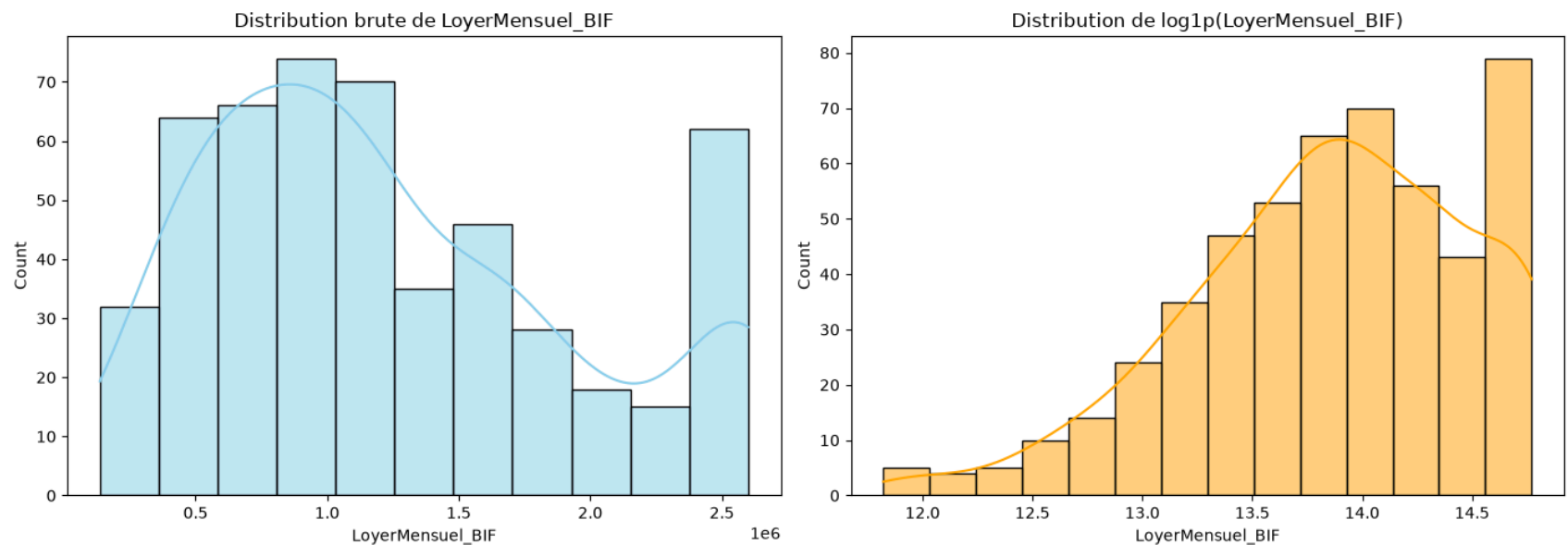
Analyse de la cible

```
In [58]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))

sns.histplot(df["LoyerMensuel_BIF"], kde=True, ax=axes[0], color="skyblue")
axes[0].set_title("Distribution brute de LoyerMensuel_BIF")

# Distribution avec Transformation Log
sns.histplot(np.log1p(df["LoyerMensuel_BIF"]), kde=True, ax=axes[1], color="orange")
axes[1].set_title("Distribution de log1p(LoyerMensuel_BIF)")

plt.tight_layout()
plt.show()
```



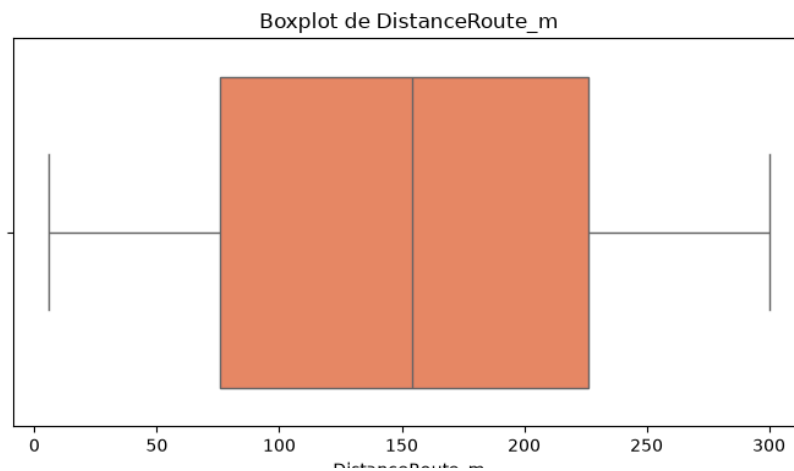
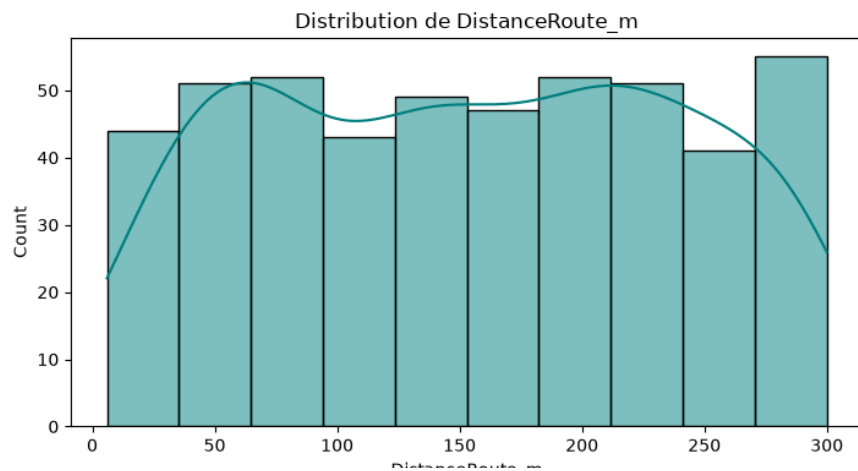
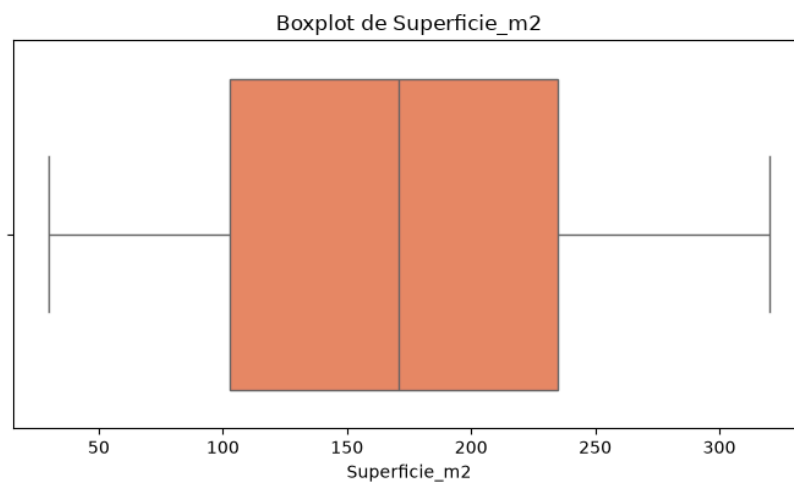
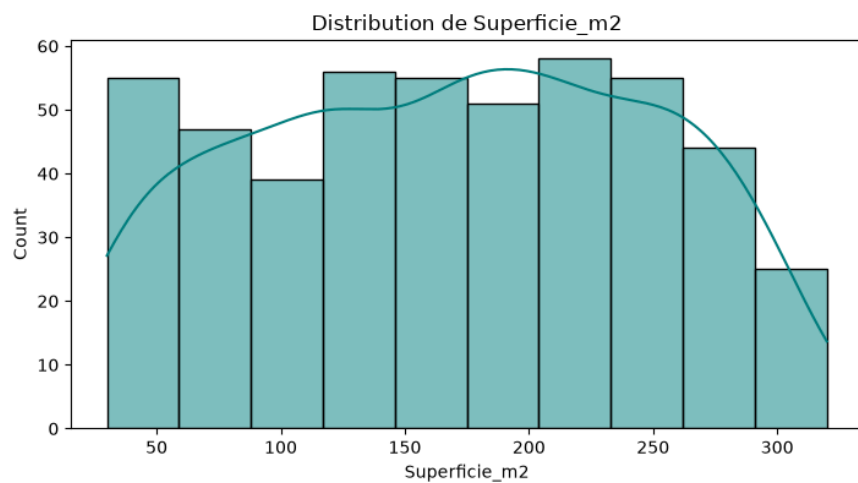
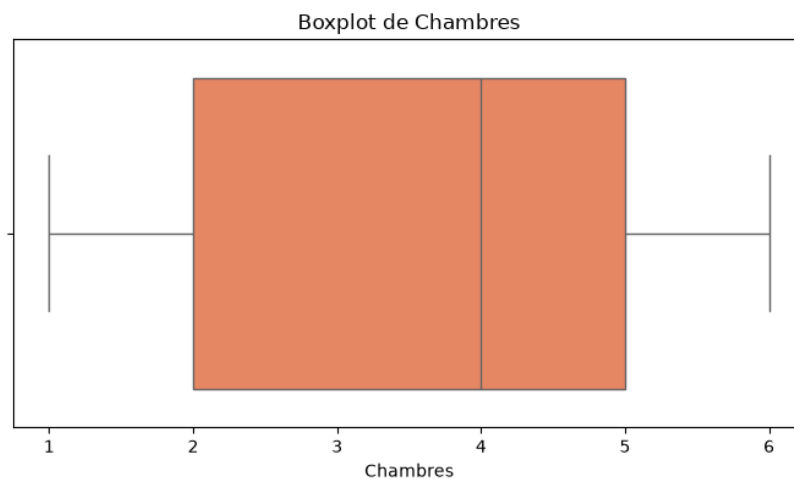
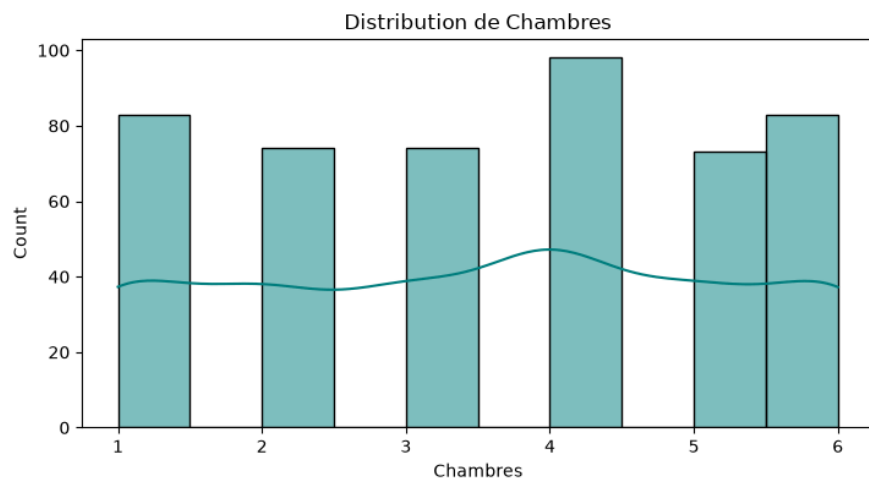
Analyses de features numériques

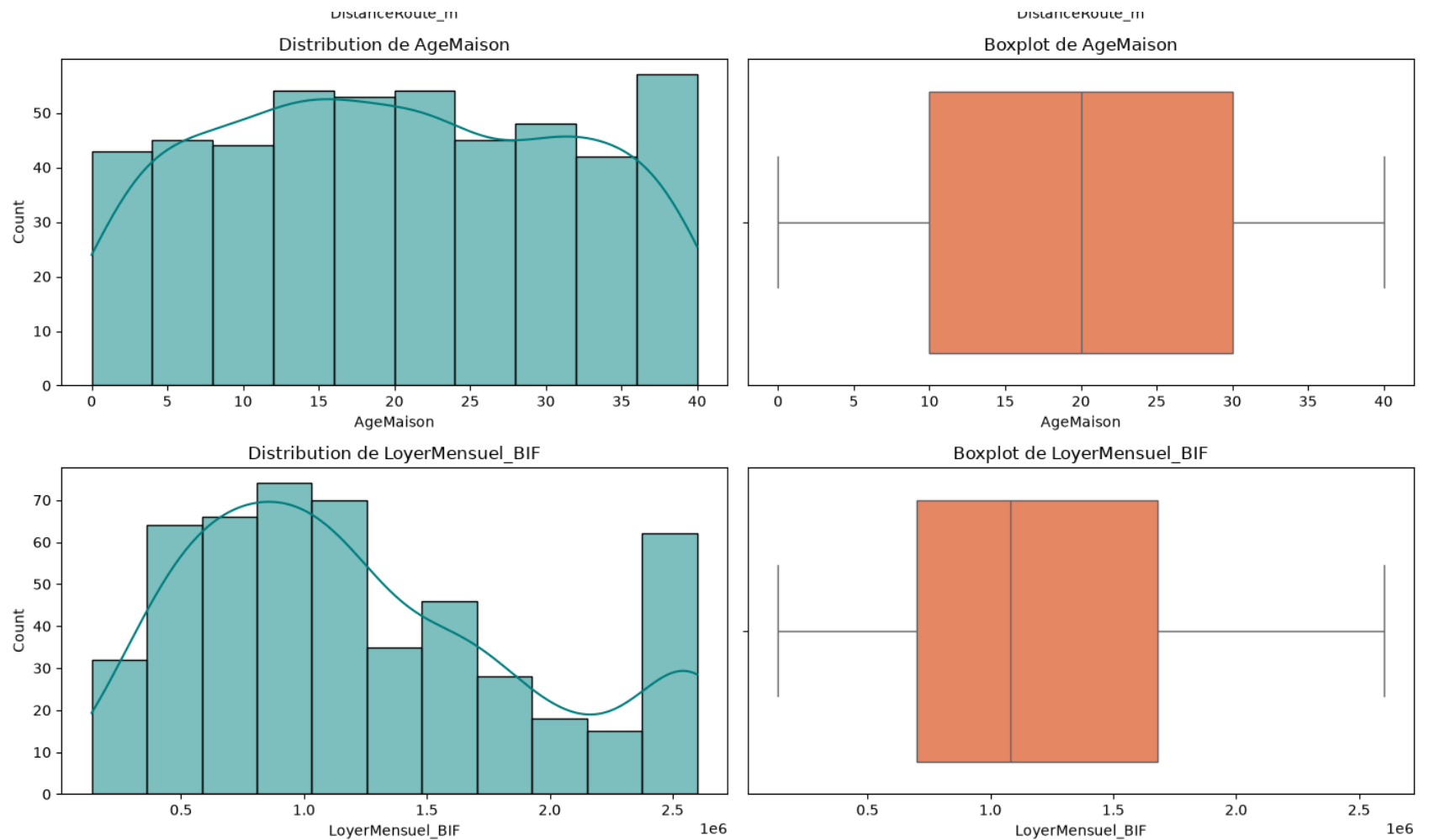
```
In [29]: fig, axes = plt.subplots(len(numeric_cols), 2, figsize=(14, 4 * len(numeric_cols)))

for i, col in enumerate(numeric_cols):
    sns.histplot(df[col], kde=True, ax=axes[i, 0], color="teal")
    axes[i, 0].set_title(f"Distribution de {col}")

    sns.boxplot(x=df[col], ax=axes[i, 1], color="coral")
    axes[i, 1].set_title(f"Boxplot de {col}")
```

```
plt.tight_layout()  
plt.show()
```





Analyses de variables catégorielles

```
In [ ]: fig, axes = plt.subplots(3, 2, figsize=(14, 12))
        axes = axes.flatten()

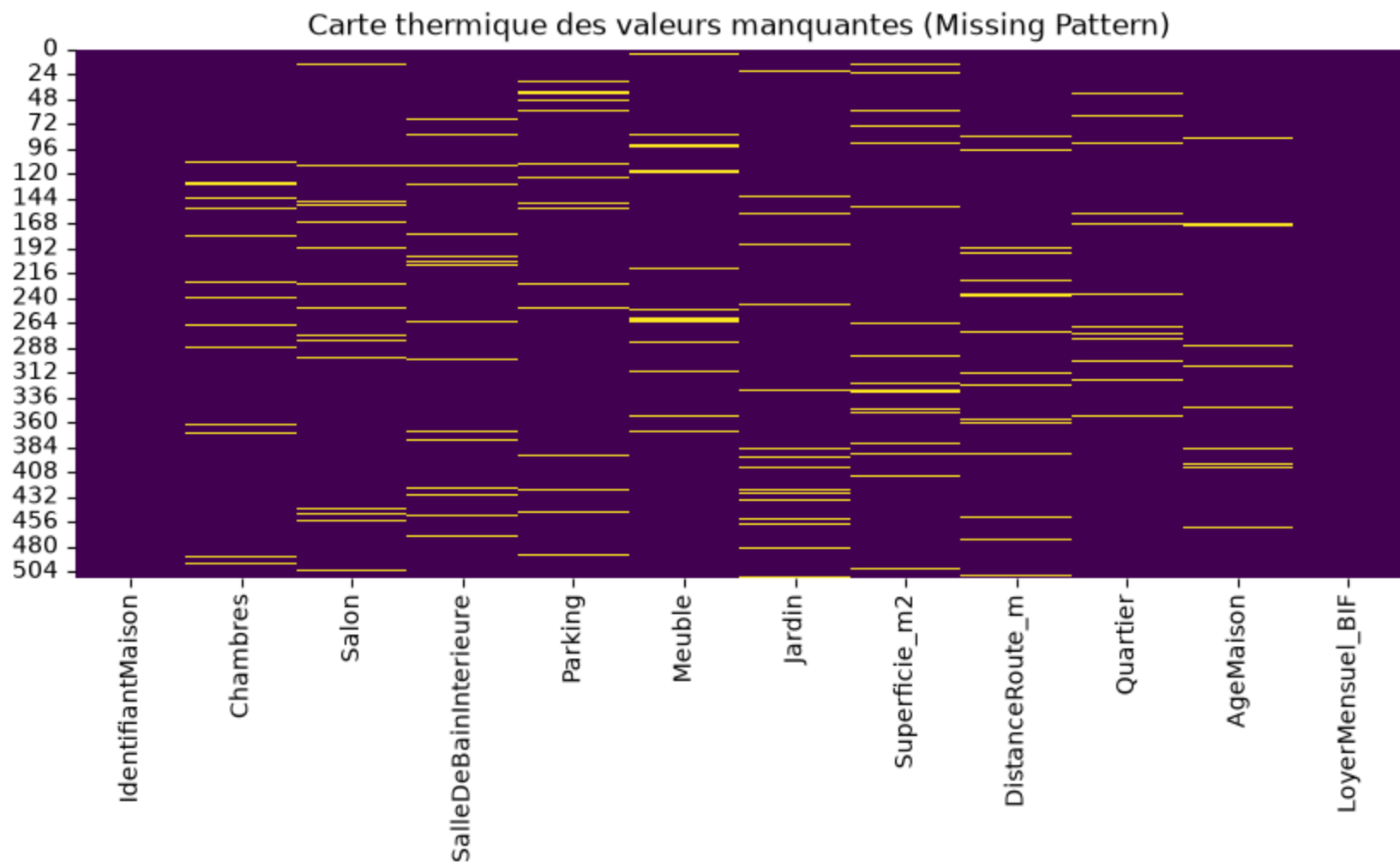
        for i, col in enumerate(category_cols):
            sns.countplot(
                data=df,
                x=col,
                hue=col,
```



```
        legend=False,  
        ax=axes[i],  
        palette="Set2",  
        order=df[col].value_counts().index,  
    )  
    axes[i].set_title(f"Répartition de {col} (Cardinalité: {df[col].nunique()})")  
    axes[i].tick_params(axis="x", rotation=45 if df[col].nunique() > 5 else 0)  
  
plt.tight_layout()  
plt.show()
```

Pattern des valeurs manquantes

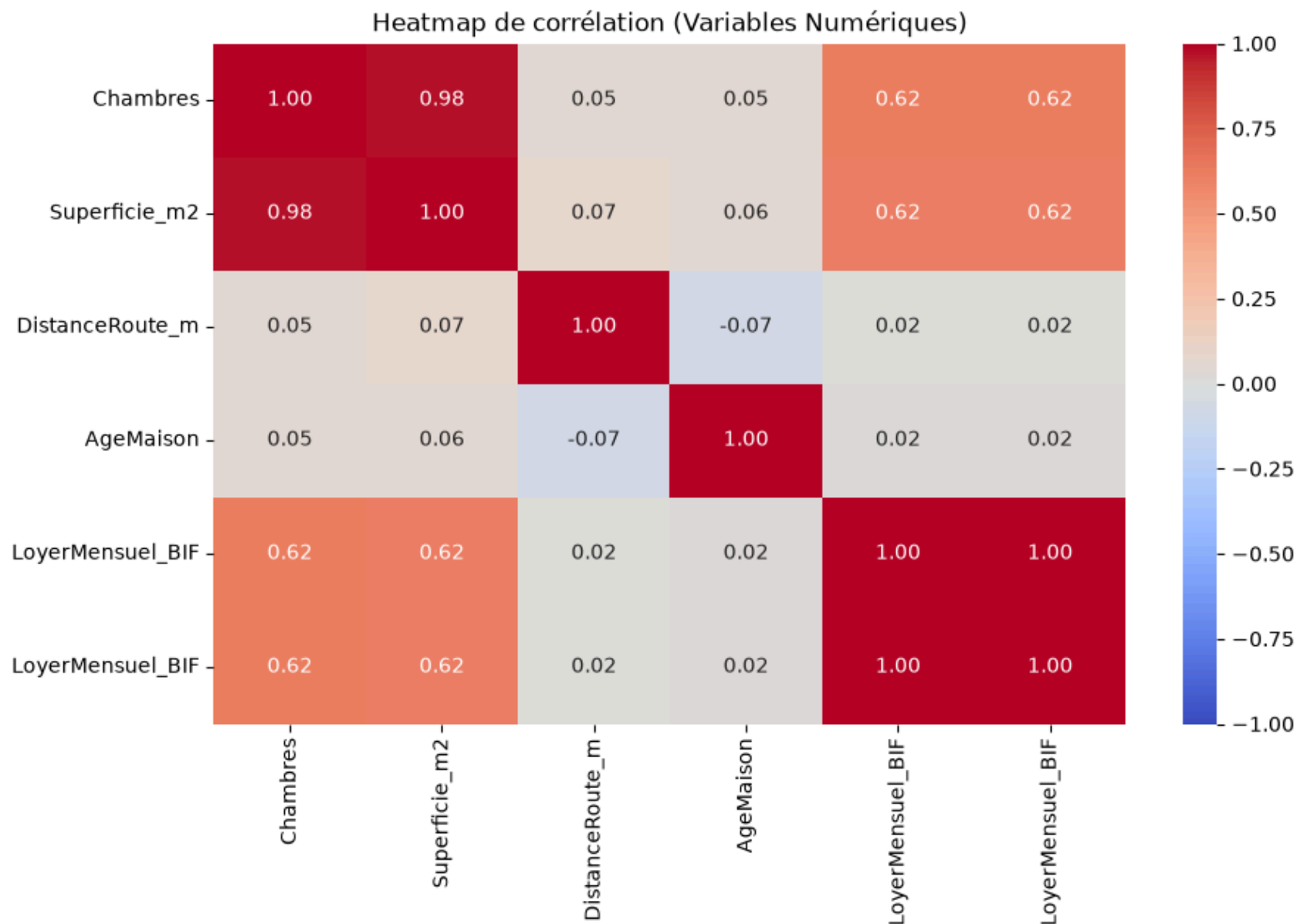
```
In [56]: # Visualiser si les manques sont corrélés entre eux (MCAR vs MNAR)  
plt.figure(figsize=(10, 4))  
sns.heatmap(df.isnull(), cbar=False, cmap="viridis")  
plt.title("Carte thermique des valeurs manquantes (Missing Pattern)")  
plt.show()
```



Corrélation avec la variable cible (LoyerMensuel_BIF)

```
In [35]: plt.figure(figsize=(1, 6))

corr_matrix = df[numeric_cols + ["LoyerMensuel_BIF"]].corr()
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", fmt=".2f", vmin=-1, vmax=1)
plt.title("Heatmap de corrélation (Variables Numériques)")
plt.show()
```



Scatter plots / Boxplots croisant chaque feature avec LoyerMensuel_BIF.

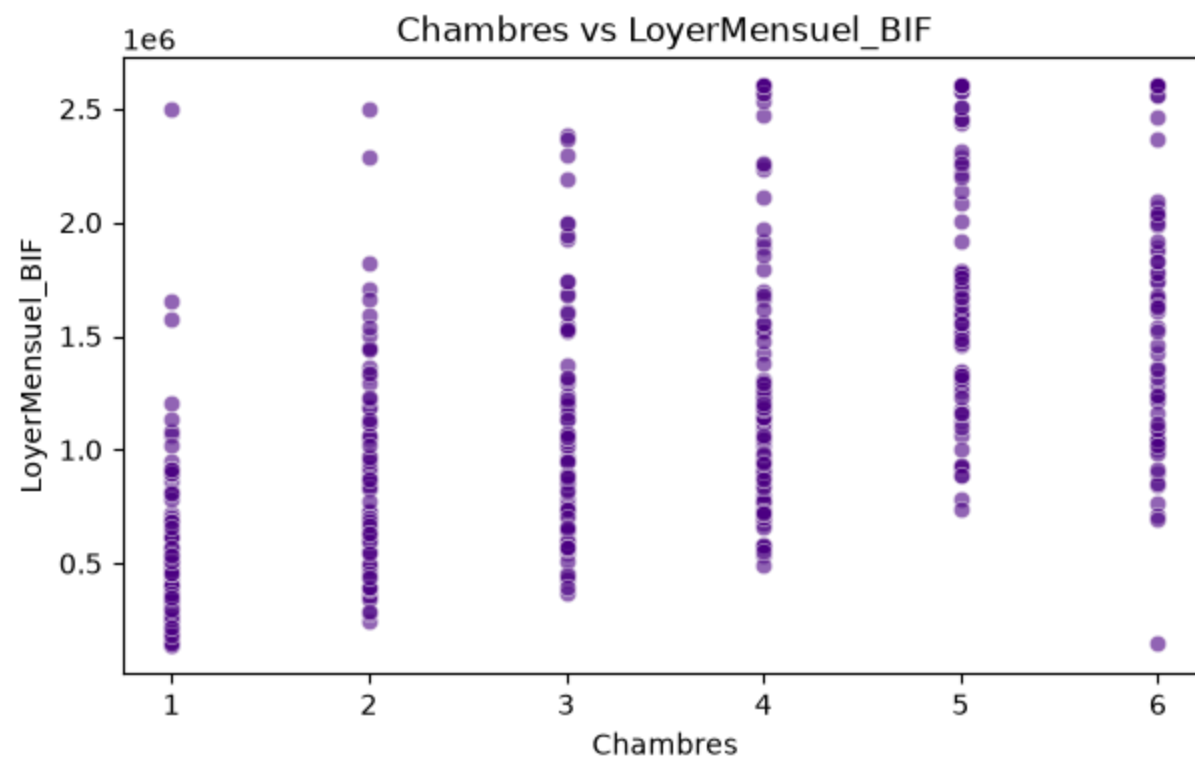
```
In [57]: # Scatter plots pour les variables numériques
         for col in numeric_cols:
```

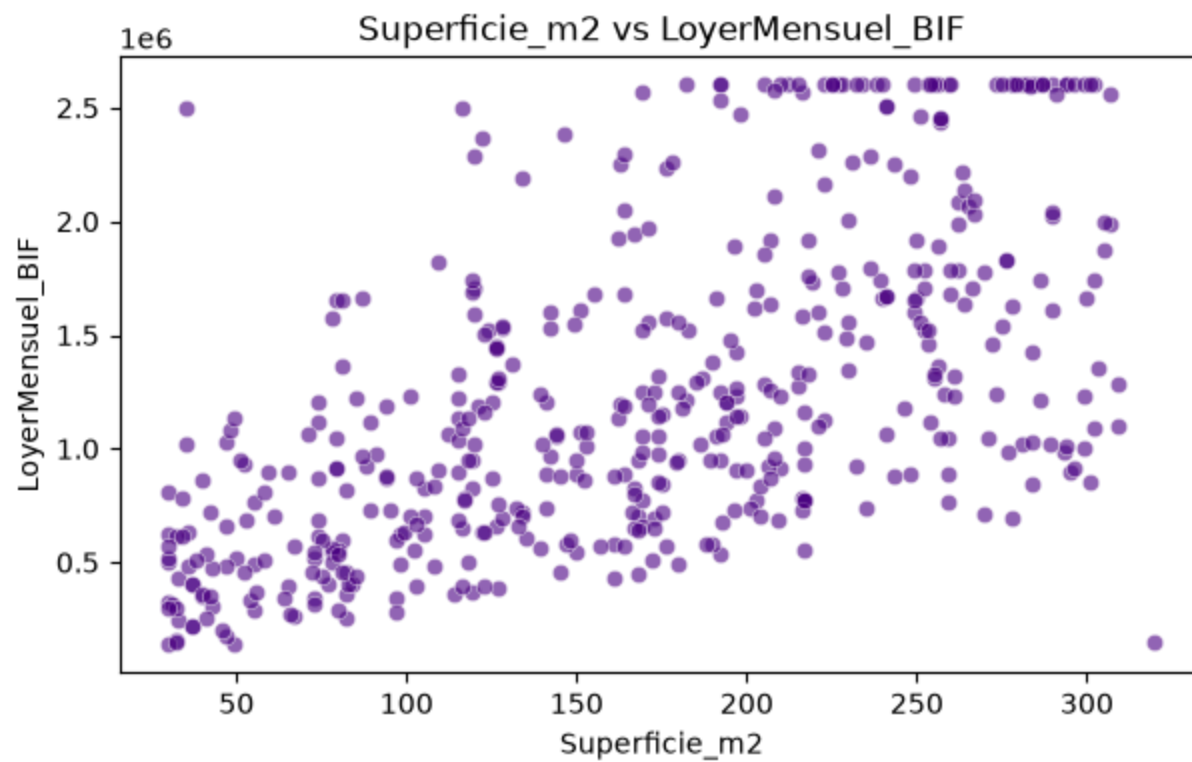
```
plt.figure(figsize=(7, 4))
sns.scatterplot(
    data=df, x=col, y="LoyerMensuel_BIF", alpha=0.6, color="indigo"
)
plt.title(f"{col} vs LoyerMensuel_BIF")
plt.show()

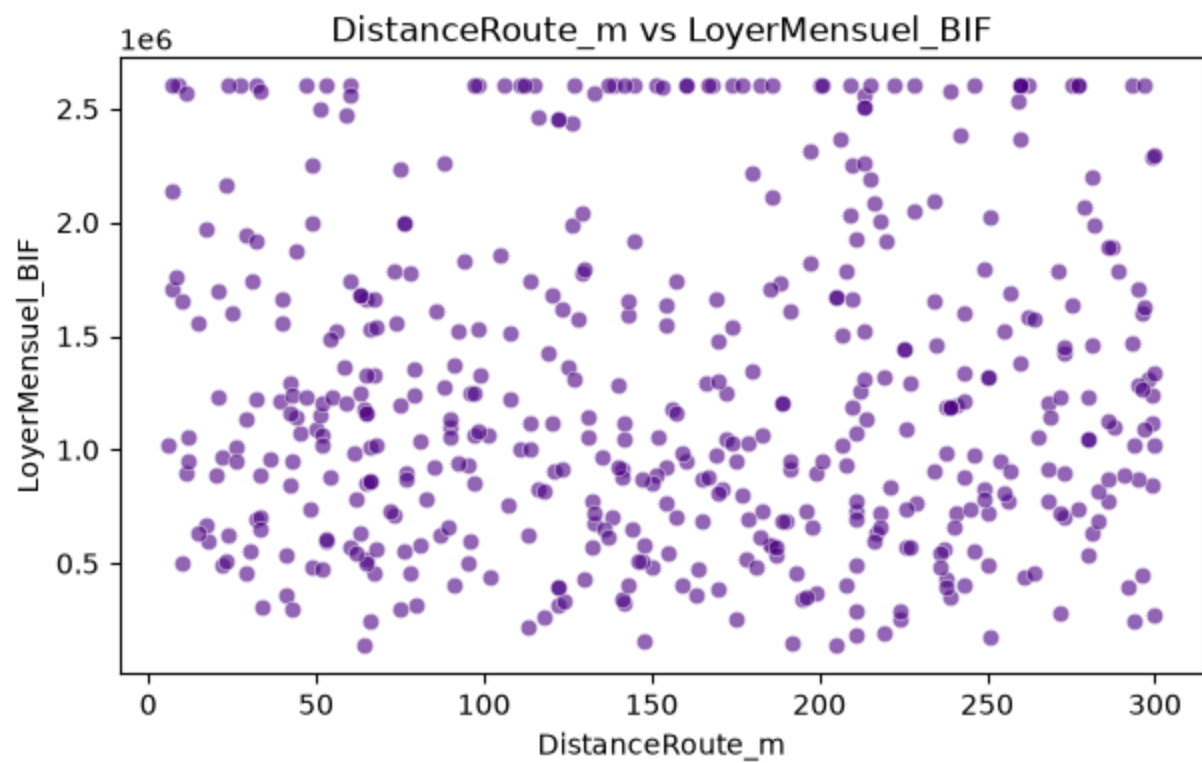
# Boxplots pour les variables catégorielles
for col in category_cols:
    plt.figure(figsize=(8, 4))

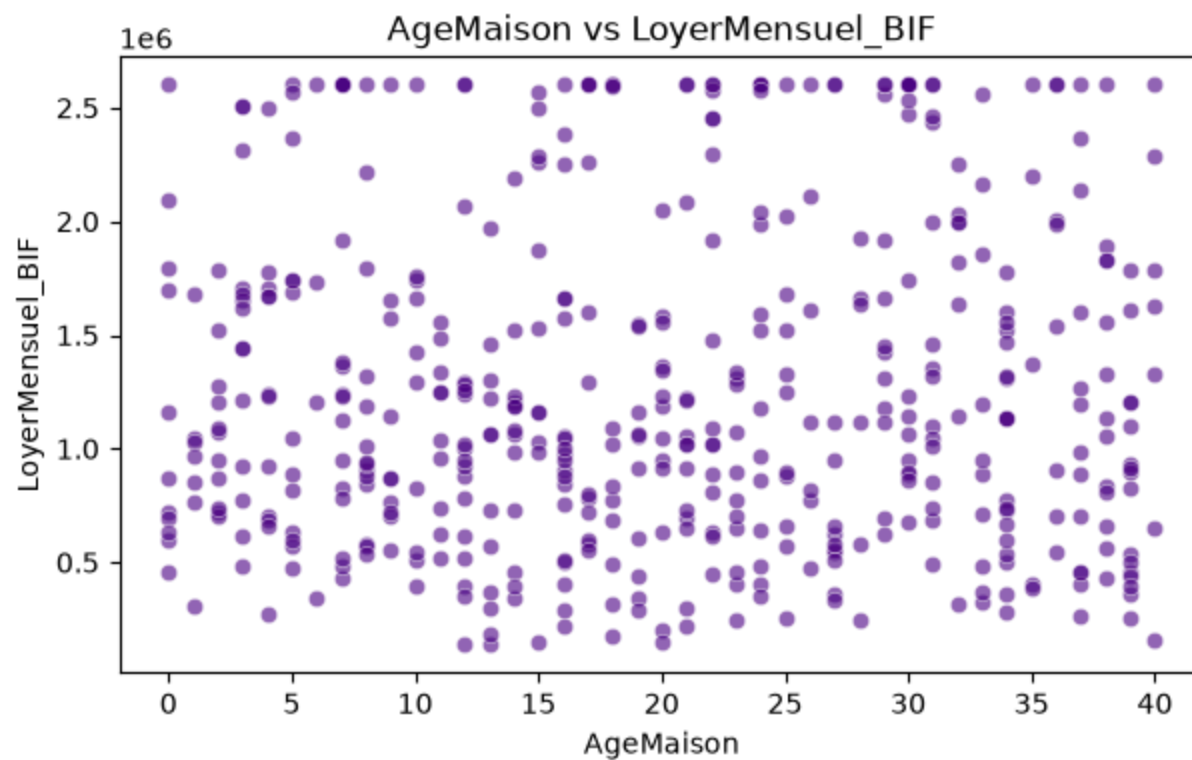
    sns.boxplot(
        data=df,
        x=col,
        hue=col,
        legend=False,
        y="LoyerMensuel_BIF",
        palette="Pastel1",
        order=df.groupby(col)["LoyerMensuel_BIF"]
            .median()
            .sort_values(ascending=False)
            .index,
    )

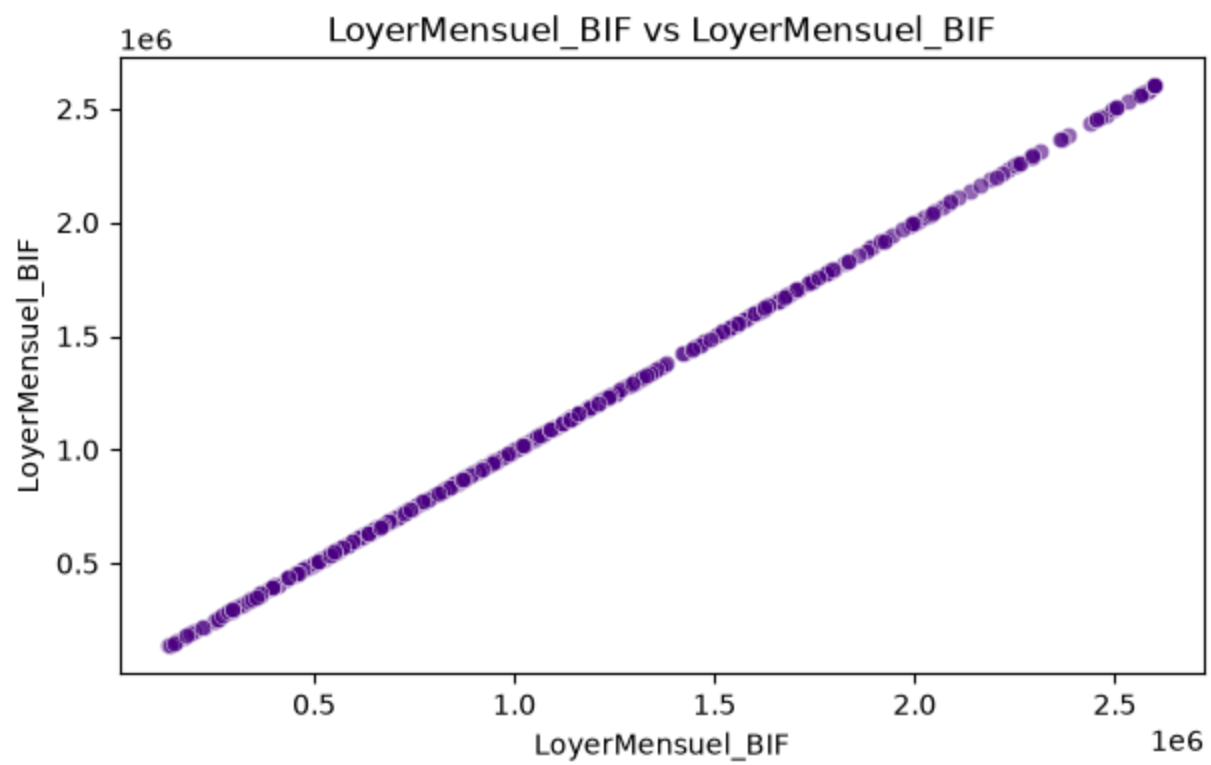
    plt.title(f"LoyerMensuel_BIF selon {col}")
    plt.xticks(rotation=45 if df[col].nunique() > 5 else 0)
    plt.show()
```

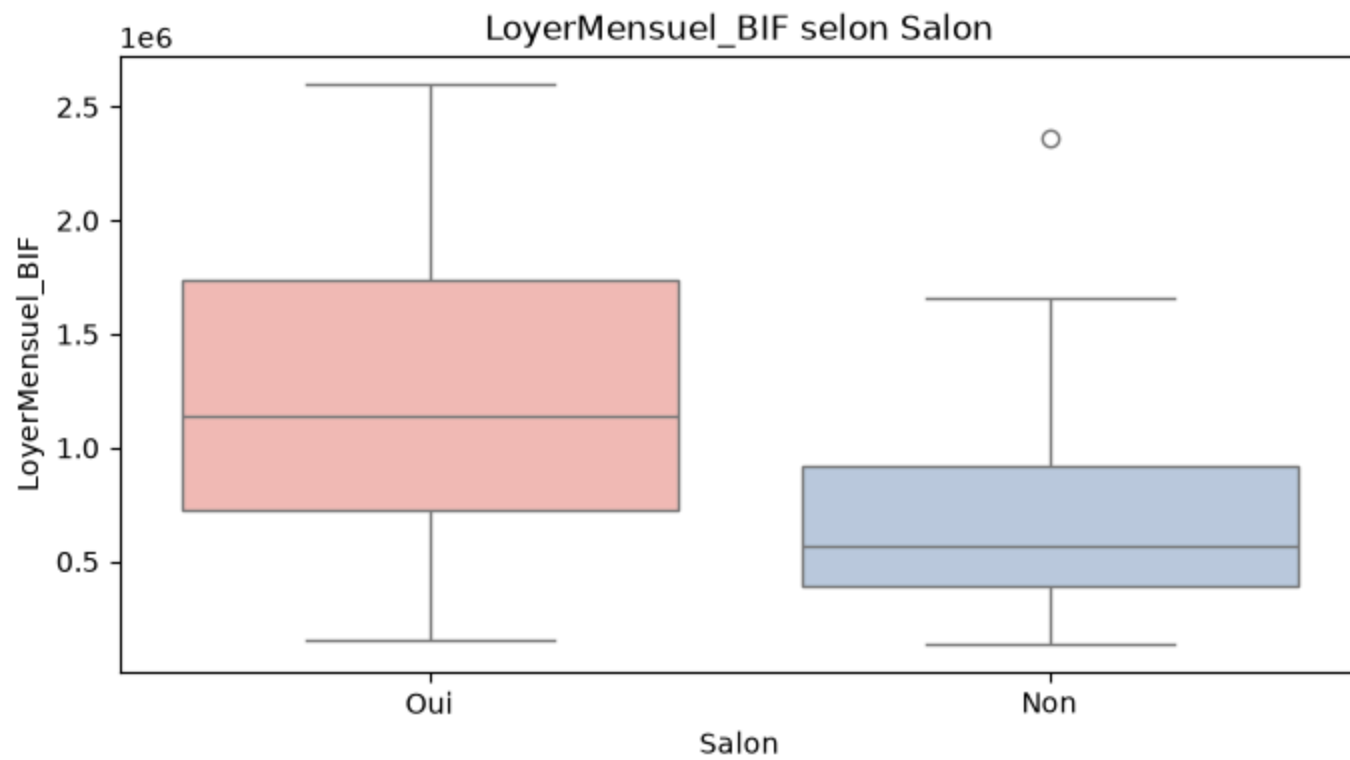


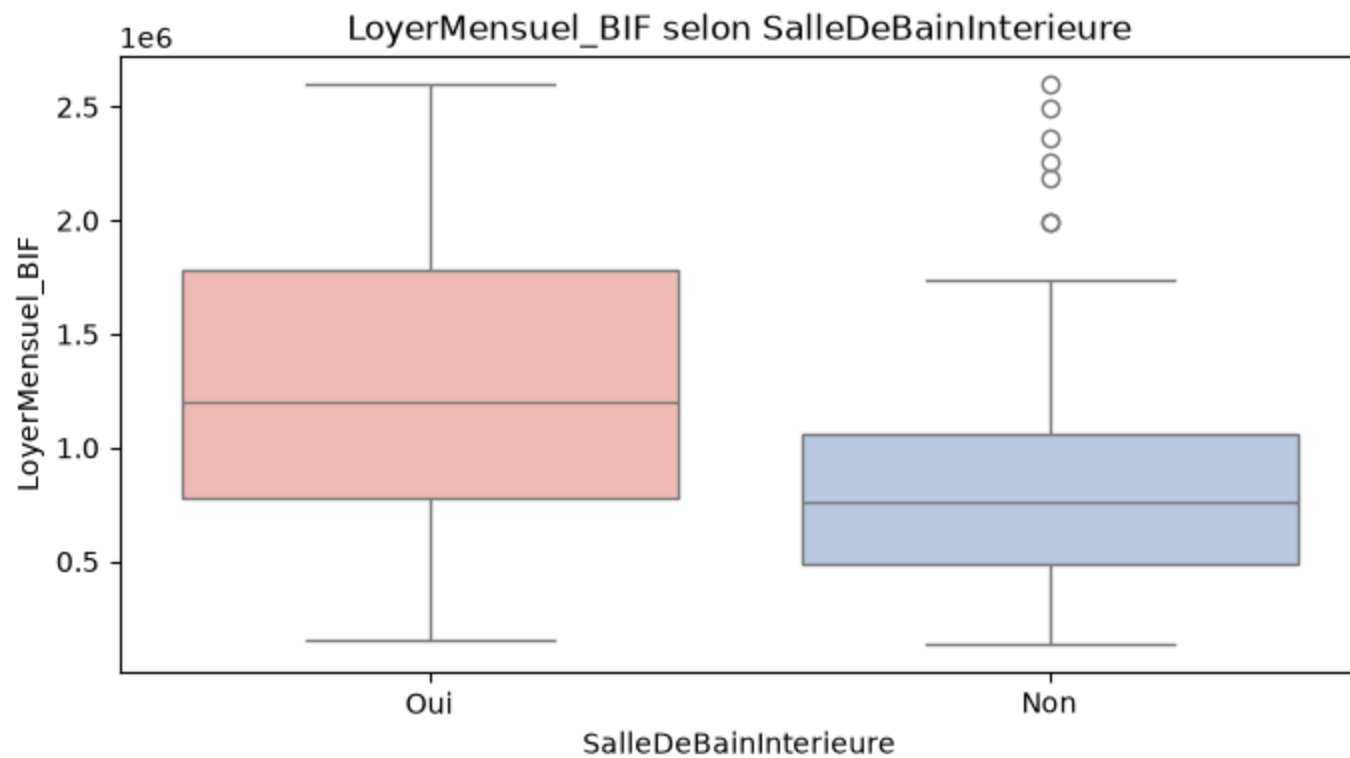


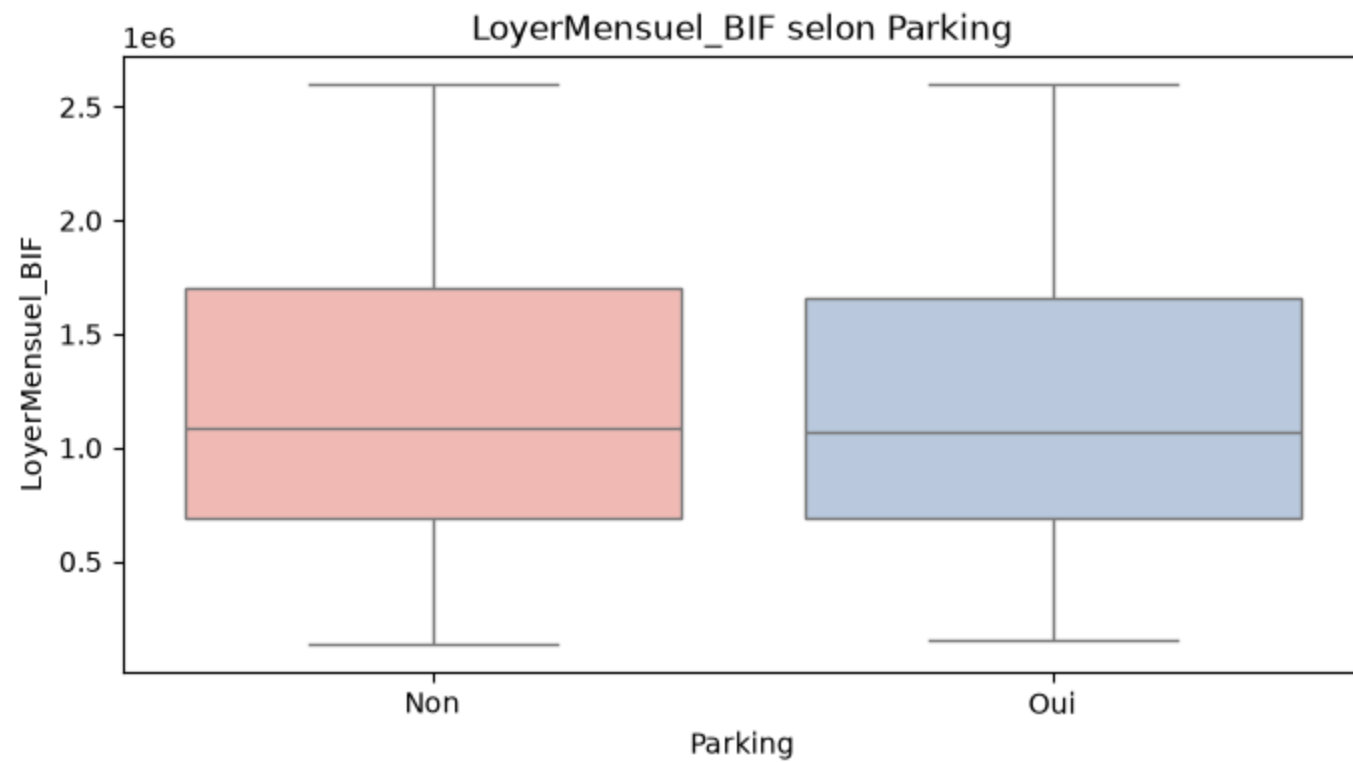


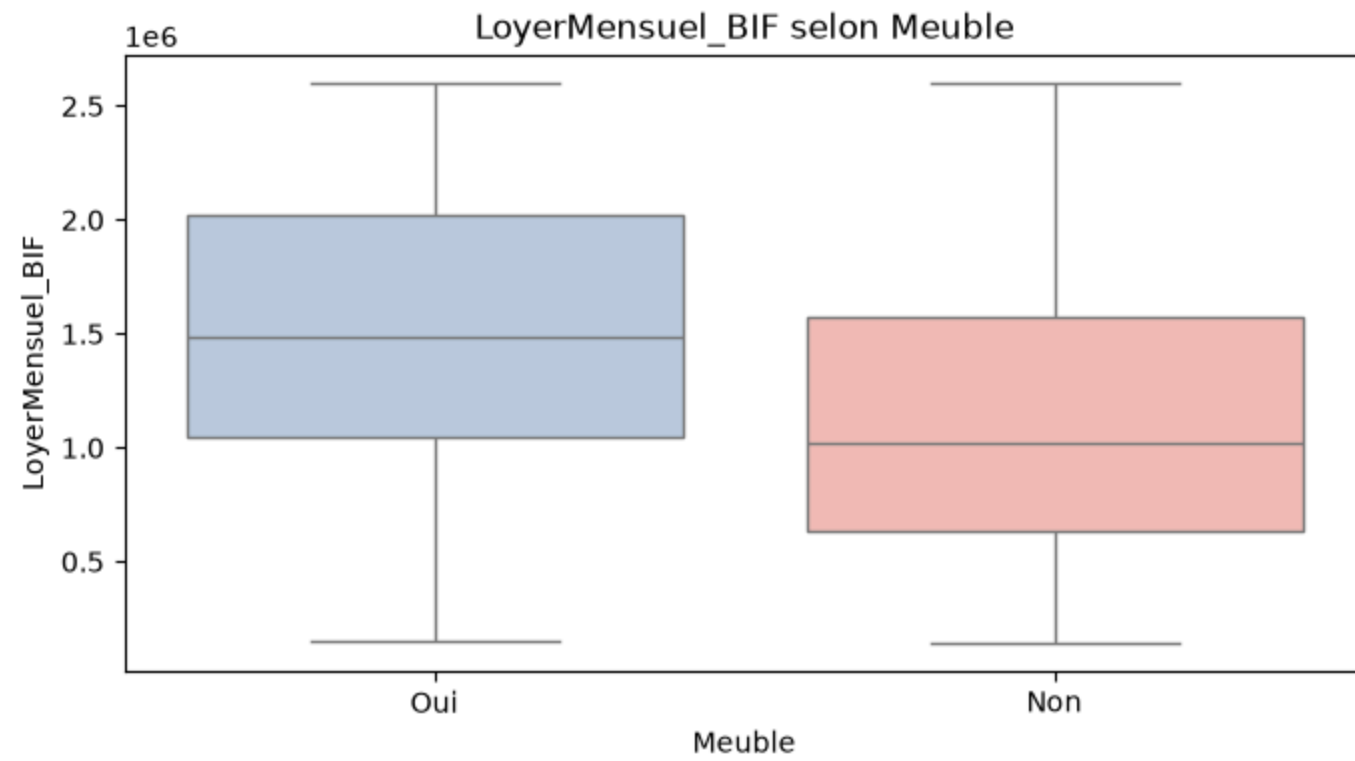


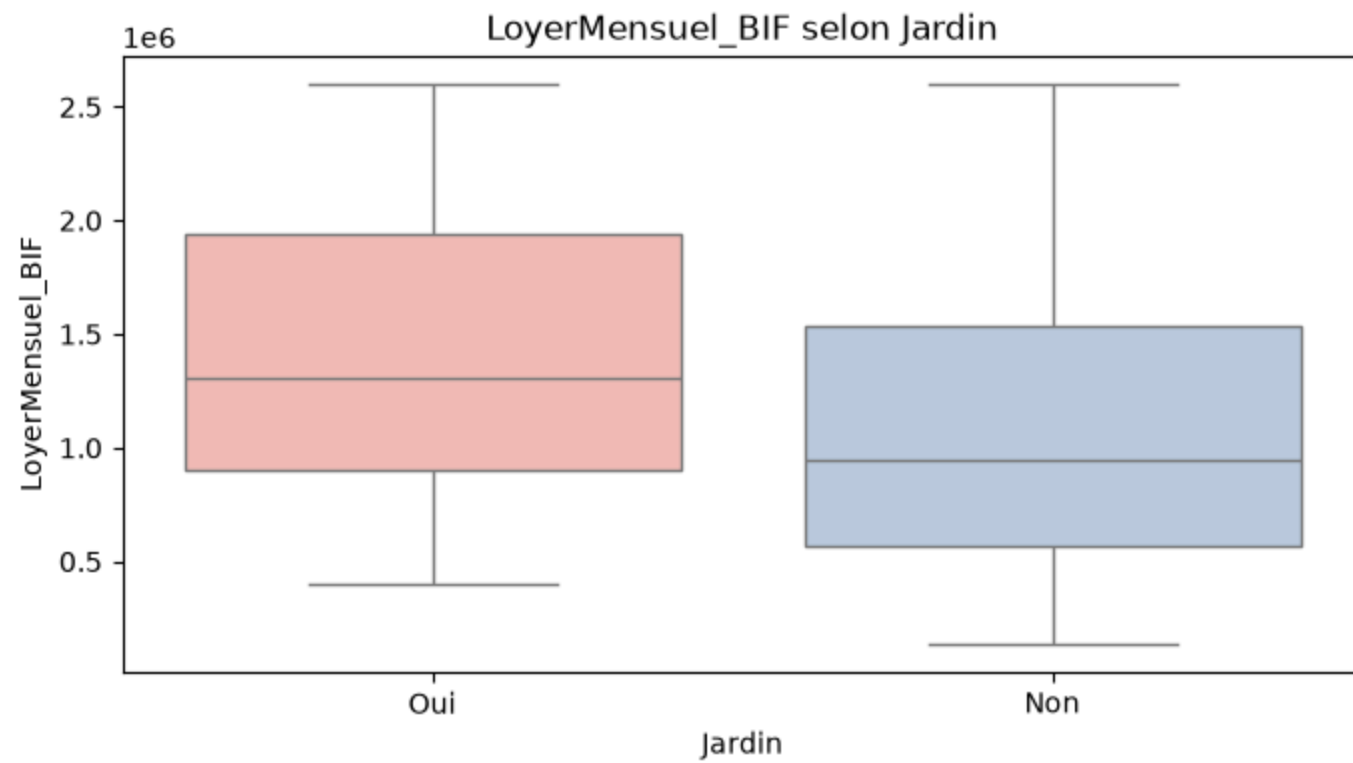


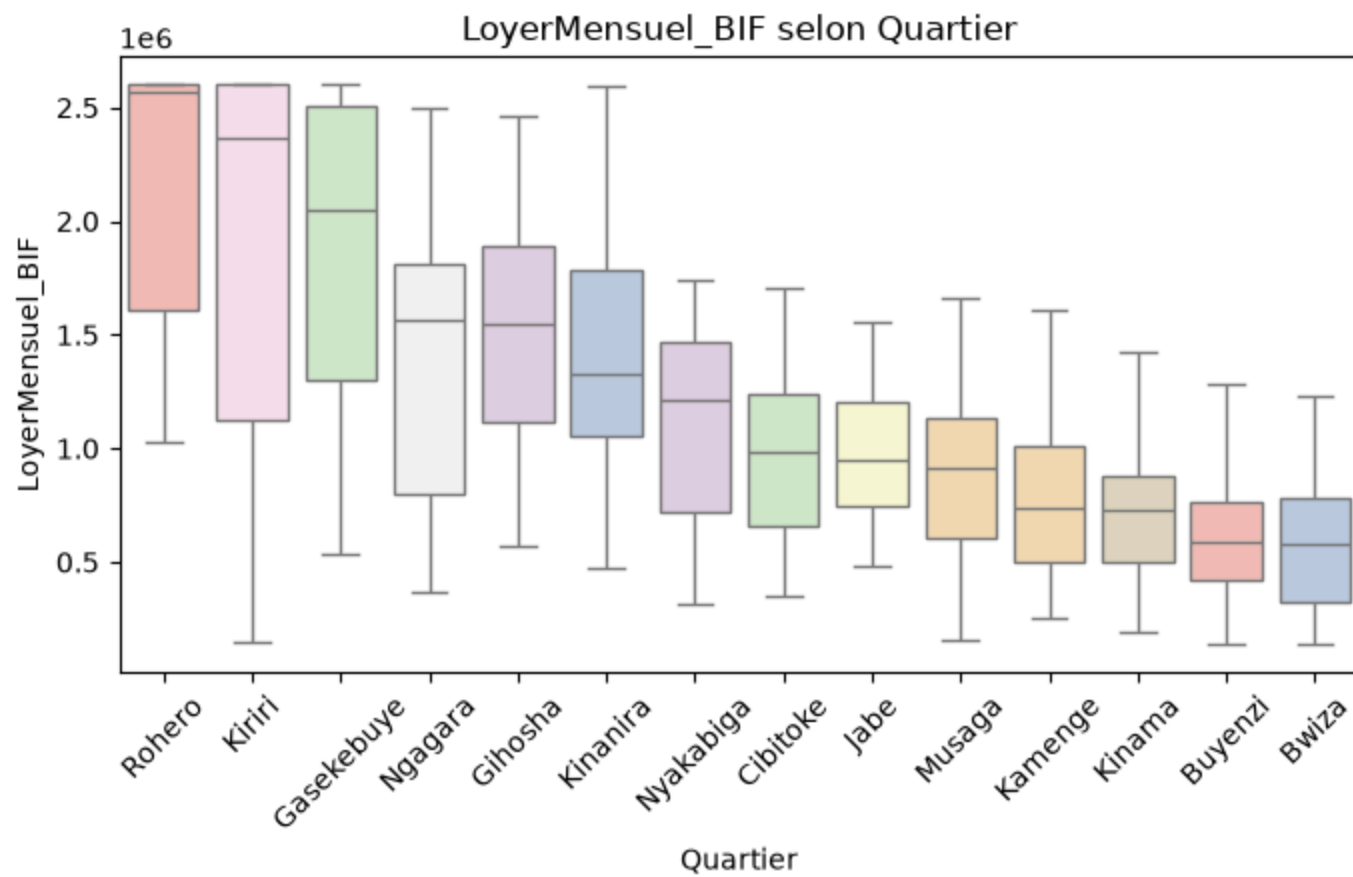












In []: